Unit 1 Foundation of database concepts

Use these model answers to check the corresponding question sheet. Other correct SQL and explanations may satisfy the same requirements.

1.1 Database components and models

Use Students, Courses and Enrolments with a composite enrolment key. Foreign keys prevent orphan enrolments; concurrency control coordinates simultaneous edits. A backup and tested recovery plan protects against data loss.

1.2 Keys, tables and design quality

Names can repeat and change. Use a stable StudentID and enforce any separate business identifiers with UNIQUE. A course cannot be recorded before a student enrols in a mixed table; repeated course names can be updated inconsistently.

1.3 SQL Server system databases

Job: msdb. Temporary table: tempdb. Template: model. Instance login metadata: master. Application data belongs in a user database.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.

Unit 2 Tools of the trade

Revision selection Original supplementary material by Lefa Malatji

2.1 Tools, SELECT and filtering

SELECT StudentID, StudentName, Age FROM Students WHERE Age BETWEEN 20 AND 24 AND StudentName LIKE 'L%' ORDER BY Age DESC, StudentID;

2.2 Aggregates, DISTINCT and grouping

SELECT City, COUNT(*) AS N FROM Students WHERE Age >= 20 GROUP BY City HAVING COUNT(*) >= 5; Age is a row condition; the minimum group size is a group condition.

2.3 Safe data changes and system functions

INSERT INTO Students (StudentID,StudentName,City,Age) VALUES (99,'Practice','Polokwane',21); UPDATE Students SET Age=22 WHERE StudentID=99; DELETE FROM Students WHERE StudentID=99; Without WHERE the statement targets every row, subject to constraints and permissions.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.

Unit 3 JOINs

Revision selection Original supplementary material by Lefa Malatji

3.1 INNER JOIN and relationship cardinality

SELECT s.StudentName,c.CourseName,e.Mark FROM Students s JOIN Enrolments e ON e.StudentID=s.StudentID JOIN Courses c ON c.CourseID=e.CourseID WHERE e.Mark>=75 ORDER BY s.StudentID,c.CourseID;

3.2 Outer joins, CROSS JOIN and self joins

The cross join creates 80 rows. SELECT a.StudentName,b.StudentName FROM Students a JOIN Students b ON a.City=b.City AND a.StudentID<b.StudentID ORDER BY a.StudentID,b.StudentID; The inequality prevents self-pairs and mirrored duplicates.

3.3 UNION and UNION ALL

SELECT StudentID FROM Enrolments WHERE CourseID=1 UNION ALL SELECT StudentID FROM Enrolments WHERE CourseID=2; Change UNION ALL to UNION to return each matching student once. A student in both courses appears twice with ALL.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.

Unit 4 Creating and altering tables

Revision selection Original supplementary material by Lefa Malatji

4.1 Schemas, databases and data types

Use NVARCHAR(100), DATE, DECIMAL(10,2) and VARCHAR(25), adjusting lengths to requirements. A phone number is an identifier with possible leading zeros, + signs and formatting; arithmetic is not meaningful.

4.2 ALTER, DROP and change planning

Find NULL emails, resolve them using a documented business rule, check length and duplicates if uniqueness is also required, test dependent code and back up. Then run ALTER TABLE with the existing type and NOT NULL in a controlled migration.

4.3 SSMS design view and generated scripts

Check the target database and schema, every column name and type, required/optional nullability, primary/unique keys, and all CHECK/default/foreign-key definitions. Save the reviewed script and test it in an empty practice database.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.

Unit 5 Constraints and referential integrity

Revision selection Original supplementary material by Lefa Malatji

5.1 Entity integrity and named constraints

NULL produces UNKNOWN in the CHECK predicate and is accepted unless NOT NULL is added. DEFAULT is used when the column is omitted or DEFAULT is specified; an explicit supplied status is accepted unless another constraint rejects it.

5.2 Foreign keys and referential actions

Keep invoices using NO ACTION or a deliberate archival policy; do not casually cascade-delete financial history. Optional assignment can use SET NULL when a staff record is removed, provided the foreign key column is nullable and business history is retained separately.

5.3 Add, remove and review constraints

First identify invalid rows and agree a correction or rejection policy. Repair the data, validate the correction, then add the CHECK with validation. Avoid hiding bad data by adding an untrusted constraint just to make deployment pass.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.

Unit 6 Normalisation and other basic design issues

Revision selection Original supplementary material by Lefa Malatji

6.1 Subqueries, EXISTS and OUTPUT

UPDATE Students SET Age=Age+1 OUTPUT inserted.StudentID,deleted.Age AS OldAge,inserted.Age AS NewAge WHERE StudentID=1; Run inside a practice transaction and roll back when finished. Do not treat OUTPUT rows as proof a transaction later committed.

6.2 Normalisation with dependency reasoning

Key is (StudentID,CourseID). StudentID determines StudentName; CourseID determines CourseName,LecturerID; LecturerID determines LecturerName; the whole key determines Mark. Use Students, Lecturers, Courses(CourseID,CourseName,LecturerID FK), Enrolments(StudentID PK/FK,CourseID PK/FK,Mark).

6.3 ERDs, object properties and denormalisation

Measure the query and execution plan first; improve appropriate indexes or query shape. If precomputed summary data is justified, define refresh timing, source of truth and reconciliation. Copying names everywhere introduces update anomalies without proving a speed benefit.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.

Unit 7 SQL Server storage, index structures and views

Revision selection Original supplementary material by Lefa Malatji

7.1 Storage hierarchy, scans and seeks

A heap stores rows without a clustered tree; the clustered index leaf contains table rows; a nonclustered index stores keys and row locators with optional included columns. A broad query may read most rows, making one scan cheaper than many lookups.

7.2 Index creation, order and maintenance

CREATE INDEX IX_Enrolments_Course_Mark ON Enrolments(CourseID ASC,Mark DESC) INCLUDE(StudentID); It may reduce reads for that pattern but adds storage and work to inserts, deletes and mark changes. Verify against actual plans and workload.

7.3 Views and update restrictions

A count row does not identify the required base student attributes or one unambiguous base row to change. Insert the complete student into Students; the view result changes when queried. Use a simple filtered view when appropriate and control permissions.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.

Unit 8 Writing scripts and batches

Revision selection Original supplementary material by Lefa Malatji

8.1 Scripts, batches and variable scope

Remove GO between declaration and use, or redeclare the variable after it. GO belongs on its own line in SSMS. Applications must split batches themselves or omit GO before sending SQL commands to the server.

8.2 IF, WHILE and CASE

CASE WHEN Mark IS NULL THEN 'Pending' WHEN Mark>=75 THEN 'Distinction' WHEN Mark>=50 THEN 'Pass' ELSE 'Fail' END. CASE takes the first true branch, so placing >=50 first would hide distinctions.

8.3 System functions and TRY CATCH

@@ERROR must be read immediately and describes one statement; TRY/CATCH groups error handling and exposes details through ERROR_* functions. A trigger may insert into another identity table and change @@IDENTITY; use SCOPE_IDENTITY() for the current scope or OUTPUT for all inserted IDs.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.

Unit 9 Stored procedures and error handling

Revision selection Original supplementary material by Lefa Malatji

9.1 Create, execute, alter and drop procedures

Use ALTER PROCEDURE dbo.usp_StudyLabStudentsByCity with the full new definition in its own batch; use DROP PROCEDURE dbo.usp_StudyLabStudentsByCity to remove it. Benefits include reusable logic and controlled execution permissions. Costs include versioning, deployment coordination and database-specific code.

9.2 Input, OUTPUT, defaults and RETURN

DECLARE @n INT,@rc INT; EXEC @rc=dbo.usp_StudyLabCountStudents @City='Pretoria',@Total=@n OUTPUT; SELECT @n AS Total,@rc AS Status; Without caller OUTPUT the assigned value is not copied back to @n.

9.3 Procedure validation and transaction errors

Test a valid student and age, an age below 18 and an unknown student. Confirm invalid cases leave data unchanged. Other sessions may change data after a precheck; database constraints enforce integrity at the actual write, so validation complements rather than replaces them.

Native T SQL tasks use SQL Server and SSMS in a disposable practice database. Browser tasks use the documented SQLite compatibility subset.